# Claude Multi-Account Fine Tuning

Provider Aliases — User Guide


1. Prerequisites
2. Quick start
3. Commands
   Useful flags
4. How a provider alias is named and what it runs
   4.1 Model tier — strongest paid if you have credit, strongest free if you don't
   4.2 Kimi Code twins (`kimi-<id>`)
   4.3 The `kimi-*` → `kc-*` rename (one-time, breaking)
5. Config dirs, plugins, and shared state
   Cross-alias session visibility (v1.5.0+)
   Multi-alias system (v1.6.0+; free-tier-first default since v1.26.0)
6. Overrides — `scripts/providers/overrides.json` `scripts/providers/key-aliases.json`
7. Verification — what `sync` actually checks (v1.14.0+)
8. Known limitation — default session color
9. Troubleshooting
   9.1 Vendor-prefixed model ids (fixed in v1.26.7)
   9.2 The launch token guards and your plugin surface
10. Remote distributed deployment & heavy testing
11. Command quick-reference
12. Individual provider notes
   Kimi Code — OAuth subscription (kc-for-coding, kc-k3, kc-k2p7, kc-for-coding-highspeed)
   z.ai Coding Plan (zai-coding-plan)
   Xiaomi MiMo (xiaomi)
   OpenCode Zen (provider id `opencode`, alias `opencode-zen`)
   Chutes (chutes)
   Poe (poe)
   HelixAgent (helixagent) — local single-GPU model
   llmctl (local, multi-profile) — one alias per RUNNING model server
13. Releasing — the mandatory live pre-release gate
13. HelixCode / HelixLLM Providers (Native HelixCode Integration)
   Key Differences from claude-providers
   HelixCode Provider List (15+ Providers)

HelixCode Model Selection Strategy
HelixCode Verification — LLMsVerifier Integration
HelixCode CLI Usage
HelixCode Server API
Provider Capabilities Matrix
Cloud Gate (W2c-1) — Local-First Default
LLMsVerifier Single Source of Truth (CONST-036)
Anti-Bluff Guarantees (CONST-035, CONST-037, CONST-038)
Integration with claude-providers
Configuration Files
Migration from Hardcoded Lists
14. Quota and limits reporting (`quota` / `limits`)
Flags
Coloring
Honest absence vs. probe failure
Account-level block
De-duplication
Example: a funded provider
Example: honest absence — no documented endpoint, and a native account
Performance
15. Decision endpoints (Jev) — non-launchable records
Governance context: hub engine, validation gate, decision models

# Provider Aliases — User Guide

`claude-providers` turns every LLM API key you keep in your keys file into its own Claude Code alias, pointed at that provider's strongest model. It reuses the multi-account toolkit's machinery (shared plugins/history, automatic `.bashrc`/`.zshrc` wiring) so a provider alias behaves like a normal `claudeN` account, just talking to a different model backend.

It is **fully dynamic**: the provider list, base URLs, transports, and model IDs are all derived at run time from your keys file + the models.dev catalog. Nothing is hardcoded. Re-run it any time to pick up new keys or refresh existing providers.

---

## 1. Prerequisites

| Need | Why | Install |
|---|---|---|
| The toolkit installed | provides `lib.sh`, the alias file, sync-state | `bash scripts/install.sh` |

| Need | Why | Install |
|---|---|---|
| A keys file | one `export NAME_API_KEY=...` per provider | `~/api_keys.sh` (default) or pass `--keys-file` |
| `jq`, `python3`, `curl` | catalog parse + resolve + fetch | system package manager |
| `claude-code-router` *(only for routed providers)* | translate Anthropic↔OpenAI/Gemini | `claude-ccr-build` — builds the **bundled Go** router from the flat `claude_code_router` checkout (found by `scripts/resolve_deps.sh claude_code_router`: `CT_CLAUDE_CODE_ROUTER_DIR`, the parent hub, `<root>/submodules/`, or a sibling clone — it is not a submodule of this repo). Do **not** `npm install -g @musistudio/claude-code-router`; that is a different binary the toolkit treats as a doppelgänger (it lacks the `ccr restart` subcommand every route-apply needs). |
| the `llms_verifier` (LLMsVerifier) checkout built *(optional)* | full "does this model work?" verification | see §7 |

Your keys file is the **only** place secrets live. `claude-providers` reads the variable *names* from it (without executing it) to discover providers, and the launch wrapper reads the *value* at the moment you start a session. No key is ever written into the repo, the alias file, or the per-provider env files.

# 2. Quick start

```
claude-providers sync          # discover + create/refresh all
        provider aliases
source ~/.local/share/claude-multi-account/aliases.sh   # or open
        a new shell
claude-providers list          # see what you got
deepseek                       # launch Claude Code on DeepSeek's
        strongest model
```

# 3. Commands

| Command | Does |
|---|---|
| `claude-providers sync` *(default)* | Discover every LLM key, resolve it via models.dev, verify it with live sentinel + tool-calling probes (see §7), and create/refresh one alias per provider. Idempotent — safe to run repeatedly. |
| `claude-providers list` | Table of **verified** provider aliases only (safe to launch): alias name, provider, transport, strong + fast model. |
| `claude-providers list-all` | Every installed provider alias, any status. |
| `claude-providers list-faulty` | Only aliases with an issue (`failed` / `unverified` / `pending`) — the filtered-out set. |
| `claude-providers show <id>` | Print the resolved env file for one provider. |
| `claude-providers remove <id>` | Remove the alias + env file; back up and remove the config dir. |
| `claude-providers add --from-key VAR --id PROVIDER` | Register a key→provider mapping (when a key's name doesn't match models.dev), then sync. |

### Useful flags

- `--keys-file PATH` — use a different keys file (default `~/api_keys.sh`).
- `--no-verify` — skip verification; create aliases regardless.
- `--offline` — don't fetch models.dev; use the local cache (errors if none).
- `--dry-run` — print what would change; write nothing.

# 4. How a provider alias is named and what it runs

- **Alias name** defaults to the models.dev provider id (`deepseek`, `groq`, `mistral`, …). Want something shorter like `dseek`? Set it in `scripts/providers/overrides.json` (see §6).
- **Strong model** → Claude Code's main model (`ANTHROPIC_MODEL`). Chosen as the provider's most capable: reasoning-capable first, then newest, then largest context — **within the tier your account can actually pay for** (see §4.1).
- **Fast model** → Claude Code's background model (`ANTHROPIC_SMALL_FAST_MODEL`), chosen as the cheapest tool-call-capable model in that same tier.

- **Transport**:
  - `native` — provider speaks the Anthropic API directly (models.dev `npm` is `@ai-sdk/anthropic`). The alias runs `claude` with `ANTHROPIC_BASE_URL` / `ANTHROPIC_AUTH_TOKEN` / `ANTHROPIC_MODEL` set.
  - `router` — provider is OpenAI-compatible / Gemini. The alias launches through `claude-code-router` (`ccr default-claude-code`), which translates the protocol.

## 4.1 Model tier — strongest paid if you have credit, strongest free if you don't

Model selection is **credit-aware**. This is a hard rule applied to every provider alias:

| Credit state of that provider's account | What the alias runs |
|---|---|
| Credit / purchased tokens available | the strongest **paid** model the provider serves that passes verification |
| No credit | the strongest **free** model (free tier / `$0` cost) that passes the same verification |
| Unknown — can't be determined | treated as *no credit*: the free choice |

**Why "unknown" falls back to free.** The two possible mistakes cost very different amounts. Choosing a paid model on an unfunded key gives you a 402/403 at launch and a dead alias right when you wanted to work. Choosing a free model on a funded key only costs some capability, and the next `claude-providers sync` promotes it as soon as the credit signal becomes readable. So an inconclusive billing probe, a provider with no balance endpoint, or a stale offline catalog all resolve conservatively.

**It's a floor on cost, not a cap on quality.** Inside whichever tier applies, the pick is still the *strongest* model. The rule narrows the candidate set; it does not lower the bar within it. It applies to both the strong and the fast slot, and to every alias `sync --multi` generates.

**Verification does not relax for free models.** A free model still has to pass the sentinel probe and the tool-calling probe (§7) before its alias is activated. "Cheapest that works" never degrades into "cheapest".

**Your pin always wins.** A `strong_model` / `fast_model` entry in `scripts/providers/overrides.json` (§6) overrides the automatic choice completely. Use it when you know exactly what you're paying for on a given provider and want the tier logic to stay out of it.

> The enforcement mechanism (in `providers_resolve.py`, with matching detection in the bundled LLMsVerifier) landed alongside this section. The table and the rules above are the **behavioural contract**; check the source for the current flag and field names.

## 4.2 Kimi Code twins (`kimi-<id>`)

For every provider id that verifies, `sync` also emits a **Kimi Code twin** that opens the `kimi` CLI against the **same backend** and the same key:

```
alias deepseek="cma_run_provider deepseek"           # Claude Code
alias kimi-deepseek="cma_run_kimi_provider deepseek" # Kimi Code,
        same backend
```

See the family-dispatch diagram `docs/diagrams/kimi-family.{mmd,svg}` for the `claudeN/kimiN/<id>/kimi-<id>/kc-<id>` naming contract at a glance.

- The no-prefix alias is **unchanged** and always opens Claude Code.
- `kimi-<id>` opens Kimi Code via a per-id `~/.kimi-prov-<id>/config.toml` (provider type by base shape, `base_url`, key from the same env record, `max_context_size` from the same derived limits) and launches `KIMI_CODE_HOME=… kimi -m "<host>/<strong-model>"`.
- Verification is **shared**: the `status.json` verdict gates both twins. A `kimi-<id>` alias is refused unless its id is `verified` (or `--force`).
- Emit them with `--kimi-aliases` (default), or suppress with `--no-kimi-aliases`.
- `kimi-*` ids (e.g. `kc-*` after the rename) get no kimi twin — see §4.3.

## 4.3 The `kimi-*` → `kc-*` rename (one-time, breaking)

The legacy aliases that opened **Claude Code** against a **Kimi-native backend** are renamed **once** (automatically at `sync`, or via `kimi-providers migrate-names`): `kimi-for-coding`→`kc-for-coding`, `kimi-for-coding2`→`kc-for-coding2`, `kimi-for-coding-highspeed`→`kc-for-coding-highspeed`, `kimi-k3`→`kc-k3`, `kimi-k2p7`→`kc-k2p7`. The `kimi-` prefix now means only "Kimi Code CLI agent". The rename is idempotent (second run is a no-op) and reversible via `backup_and_remove` backups. Update any scripts or muscle memory that reference the old aliases.

# 5. Config dirs, plugins, and shared state

Each provider gets `~/.claude-prov-<id>`, which symlinks the same shared items as your accounts (`projects`, `todos`, `plugins`, `settings.json`, `CLAUDE.md`, …). So **every plugin you have in `claude1..N` is available in every provider session**, and history/projects are unified. `sync` also force-enables the always-on set in shared settings: **superpowers, systematic-debugging, frontend-design, code-review**.

Provider dirs are deliberately **excluded from account auto-detection**, so they never get merged into your real accounts' auth/identity and never interfere with `claude-unify` or `claude-add-account`. Your existing `claudeN` aliases keep working exactly as before, and you can still create new accounts the same way.

## Cross-alias session visibility (v1.5.0+)

Sessions created under **any** alias — `claude1`, `claude2`, `deepseek`, `opencode`, `xiaomi`, or any other — are visible from **every** other alias via `/resume`. This works automatically:

1. **Before launch**: `claude-sync-state pull` merges every account's and provider's `.claude.json` into the launching dir. This includes `lastSessionId` (what `/resume` uses), `allowedTools`, MCP config, and all other project settings.
2. **After exit**: `claude-sync-state push` merges the post-session state back out, so the next alias to launch picks up the new session.
3. **Session files** are in `~/.claude-shared/sessions/`, which every alias symlinks to — so the session data itself was always shared; the new work ensures `.claude.json` project metadata is also merged.

**Example workflow:**

```
# Start a project as deepseek
cd /path/to/my-project
deepseek          # bare launch: auto-creates the project's
        session
# ... work for a while, then exit

# Continue the same project as opencode-zen
opencode-zen      # bare launch: AUTO-RESUMES the same project
        session — no /resume needed
```

A **bare** alias launch (no arguments) now auto-resumes — or first-time creates — the one long-lived session keyed to the project root, so switching aliases continues the same conversation

automatically (see §12 of `../Claude_Multi_Account_Fine_Tuning.md` and `SESSION_COLOR.md`). You only need `/resume` to switch to a **different** session than the project's own.

**Performance:** adds ~1-2 seconds per launch (jq merge across all dirs). Same overhead that `claudeN` aliases already have.

**Note:** provider dirs are still excluded from `cma_detect_accounts` (used by `claude-unify` and `claude-add-account`). Only `claude-sync-state` includes them, so the existing account management is unaffected.

## Multi-alias system (v1.6.0+; free-tier-first default since v1.26.0)

`sync` creates **one alias per provider** using the 2 strongest models, and then (v1.26.0+, ATM-860) runs the **per-model multi-alias phase by default**: it verifies each provider's **FREE-tier models** with real completion probes and creates **multiple aliases** — `provider`, `provider2`, `provider3`, etc. — each with a different pair of strong + fast models. Free-vs-paid derives from real data only (models.dev `cost` rows, `:free` ids, self-hosted local endpoints); a model whose tier cannot be derived is treated as **paid** and skipped unprobed (fail-safe on spend), recorded honestly under `skipped_models` in `<provider>_verified.json`. Paid models are **never probed by default** — that costs real money and is an explicit opt-in.

```
# Standard (default): 1 alias per provider + per-model aliases for
        FREE models
claude-providers sync

# Opt in to probing paid/unknown-tier models too (spends real
        money)
claude-providers sync --include-paid          # or
        CMA_SYNC_INCLUDE_PAID=1

# Only the per-model phase (no single-alias sync); free-tier by
        default
claude-providers sync --multi

# Skip the per-model phase entirely (legacy single-alias-only
        shape)
CMA_SYNC_MULTI=0 claude-providers sync

# Options
claude-providers sync --multi --max-aliases 10 --min-score 20
```

Alias generation is idempotent and deterministic: ranking/pairing tie-breaks on model id, so re-running sync yields byte-identical alias env files and the same `provider`/`provider2`/... naming. Generated provider aliases live in the `~/.claude-prov-*` namespace

and launch via `cma_run_provider` — a class the account dispatcher refuses, so they can never shadow or outrank a native `claudeN` account alias.

**How verification works (v1.14.0+):**

Each model is tested via live HTTP probes against the provider's chat endpoint: a sentinel probe (sends "Reply with exactly: VERIFY_OK" and **requires** the response to contain `VERIFY_OK`) and a tool-calling probe (requires the model to actually emit a tool call). Models are scored on 7 dimensions (0-100):

| Dimension | Weight | What it checks |
|---|---|---|
| Existence + valid response | 25pts | Model exists, returns real content |
| Tool calling | 20pts | Supports function/tool_calls |
| Reasoning | 15pts | Has chain-of-thought / reasoning_content |
| Context window | 15pts | ≥8K tokens (log scale) |
| Streaming | 10pts | SSE streaming support |
| Latency | 10pts | Response under 2s (full) or 5s (half) |
| Free tier | 5pts | $0 input cost |

**Tool calling is a hard gate, not just score.** A model that answers chat but never calls tools is marked **not verified**, no matter how high its score — Claude Code is entirely tool-driven, so a tool-less model always breaks at runtime.

**Anti-bluff detection** prevents false positives:

- HTTP 200 with error body (JSON error wrapped in success)
- Empty response content
- **200 responses that lack the `VERIFY_OK` sentinel** (proxy fallback, silent model swap, canned text)
- Boilerplate "I'm unable to" responses
- Models that claim capability but don't deliver

**Alias pairing:**

- Models sorted by score, paired 2 per alias (strong + fast)
- Odd count: last model used for both positions in last alias
- Single model: used for both positions
- Default max: 5 aliases per provider (configurable)

**Verification cache:** results cached for 24h to avoid re-testing on every sync. Cache stored in `~/.local/share/claude-multi-account/providers/verification_cache.json`. The cache carries a schema version; results written by older verification logic (e.g. before tool calling was required) are ignored automatically.

## 6. Overrides — `scripts/providers/overrides.json`

Per-provider pins. Empty by default. Any field you set wins over the models.dev-derived value:

```
{
  "deepseek": {
    "alias": "dseek"
  }
}
```

This is how you (for example) give DeepSeek the short alias `dseek` — without touching any code.

A `strong_model` / `fast_model` pin here also **overrides the credit-aware tier choice** described in §4.1. If you pin a paid model on a provider whose credit state the toolkit reads as empty or unknown, that pin is honoured — the tier logic never second-guesses an explicit operator decision, so the launch failure that follows an unfunded key is then yours to expect.

> **Do not copy transport/base_url from an example.** As of **v1.19.0** every *hosted* provider — DeepSeek and Xiaomi included — is pinned to `router` transport on its OpenAI-compatible endpoint, because both were verified working there and a single uniform path is far easier to debug. Every `transport` pin in `overrides.json` today is `router`: deepseek, xiaomi, opencode, opencode-go, chutes, kc-for-coding, hyper. Two of them:
>
> ```
> "deepseek": { "transport": "router", "base_url": "https://
>         api.deepseek.com" }
> "xiaomi":   { "transport": "router", "base_url": "https://
>         api.xiaomimimo.com/v1" }
> ```
>
> Pasting an older `"transport": "native"` / `.../anthropic` block would revert that fix. Always check the live values in `scripts/providers/overrides.json` before overriding transport or base_url.

> The one shipped `native` pin left is the **local** `helixagent-native` (`scripts/providers/helixagent-native.json`), which is a separate provider record rather than an `overrides.json` entry — it is not a counter-example for a hosted provider.

**`scripts/providers/key-aliases.json`**

Maps a key variable name to a models.dev provider id, for the cases where the name differs (e.g. `CODESTRAL_API_KEY` → `mistral`, `ZAI_API_KEY` → `zai`). Use `claude-providers add --from-key VAR --id PROVIDER` to append entries.

# 7. Verification — what `sync` actually checks (v1.14.0+)

By default `sync` verifies every provider with **two live probes** against its chat endpoint, using the exact model the alias will run:

1. **Sentinel probe** — asks the model to reply with exactly `VERIFY_OK` and requires that token in the response. A 200 without it (or with an error object smuggled into the body) is a bluff: the endpoint answered *something*, not the requested model.
2. **Tool-calling probe** — requires the model to emit a real tool call (`tool_calls` / `tool_use`). Claude Code is tool-driven; a model that only chats is useless in practice.

Verdicts:

- `verified` — both probes passed. The alias is created and launchable.
- `failed` — a definitive problem: auth/billing rejection (401/402/403), model missing (404), missing sentinel, error-in-200, or no tool call. The alias is **not activated** (and the launch gate refuses to run it).
- `unverified` — the probes were inconclusive (rate-limit 429, 5xx, timeout, no network, no key). The alias is still created but the **launch gate** refuses to start it until a later sync reaches a verdict (override with `--force` — not recommended).

This replaced the old best-effort check (a bare `GET /models`), which proved only that the key was accepted — not that the model can actually run Claude Code. Both Anthropic-native (`/v1/messages`) and OpenAI-compatible (`/chat/completions`) endpoint shapes are probed in their native format.

**The verdict gates both twins.** The same `status.json` record that gates the Claude alias (`deepseek`) also gates its `kimi-deepseek` Kimi Code twin — there is one verification, two aliases sharing its outcome. A `kimi-<id>` alias is refused at launch unless its id is `verified` (override with `--force`).

For an additional authoritative layer ("can this model genuinely see and describe my code?") build the LLMsVerifier checkout. It is not a submodule of this repo: provide it flat (the parent hub's `submodules/llms_verifier`, or a sibling clone) or point `CT_LLMS_VERIFIER_DIR` at it:

```
LV="$(bash scripts/resolve_deps.sh llms_verifier)" \
  || git clone --depth 1 git@github.com:vasic-digital/
        LLMsVerifier.git ../llms_verifier
LV="$(bash scripts/resolve_deps.sh llms_verifier)"
cd "$LV" && go build -o bin/model-verification ./llm-verifier/cmd/
        model-verification
```

Once `<llms_verifier>/bin/model-verification` exists, `sync` prefers it as the existence check. `sync` also runs the two-round semantic code-visibility check (sentinel + independent judge) when it is available; billing/auth rejections there demote the alias, while transient judge/infra errors are an honest SKIP that never demotes. (Requires the Go toolchain; the build reads keys from your keys file.)

The live-TUI layer (`claude-providers verify <id> --deep`, runner `verify_superpowers_tui.sh`) additionally **proves which backend served the turn**. Router-transport aliases share one ccr `Router.default`, and an alias whose `base_url` is the gateway itself skips its own rewrite and would otherwise inherit the previous provider's route — a PASS that says nothing about the alias under test. Every evidence file now records `# ROUTE-INTENDED:` and `# ROUTE-RESOLVED:` (the latter read after the launch), and the leg fails with `# FAIL: route-mismatch` when they differ, or `# FAIL: route-unknown` when the route is unreadable. Native-transport aliases record an explicit `n/a` and are not route-checked.

Two further refusals close the remaining holes. `.Router.background` is checked alongside `.Router.default`, because Claude Code dispatches background sub-requests of the same turn through it — a mismatch there is `# FAIL: route-mismatch-background`. And because reading the config file back only proves what it *says*, the leg also demands a **restart receipt** bracketing the launch (a fresh `gateway listening on` line in `~/.claude-code-router/<alias-id>/service.log`, or a changed `service.json` in that same directory): the wrapper's `ccr restart` runs under `|| true` and can legitimately be refused, which would leave the previous provider serving while the file reads back correct. Without a receipt the leg fails closed

with `# FAIL: route-unproven`. For router-transport aliases `jq` is a hard precondition — without it the route is unreadable and the leg refuses rather than skipping.

Every ccr path above is **per alias**: each alias owns `~/.claude-code-router/<alias-id>/` with its own `config.json`, `service.json` and `service.log`. There is no single global set to inspect.

Two further refinements to what the leg *prints*, both of which matter when you are triaging rather than releasing. A timeout (rc 124) whose transcript carries `"terminal_reason":"aborted_streaming"` is now marked `# FAIL: stream-aborted (output_tokens=… duration_ms=… bound=…s)` and states that it is **not** a trust/overwrite dialog — the old message guessed a dialog for every rc-124 and was wrong for all of them on the run that motivated the change. And a layer-3 failure now quotes the driver's own `reason` instead of asserting "cannot see code / bluffed": driver exit 1 also covers definitive provider rejections (401/402/403/404) of the model under test, so the message can now say `non-200 status 402: "Insufficient balance for request."` and send you to the account rather than to the model.

# 8. Known limitation — default session color

The original goal was for each provider session to default its `/color` to **purple**. Investigation of Claude Code 2.1.195 found that `/color` is **session-scoped and TUI-only — it is never persisted to disk**, and there is no settings key, hook, or environment variable to set a default color. So this cannot be automated with the current Claude Code.

**Workaround:** type `/color purple` at the start of a provider session. `purple` is a valid color. If a future Claude Code adds a persistable color setting, `sync` will seed it automatically.

# 9. Troubleshooting

| Symptom | Cause / fix |
|---|---|
| `provider 'X' needs claude-code-router` | Routed provider, the bundled `ccr` gateway is not installed → `claude-ccr-build` (builds the vendored Go router; needs the Go toolchain). |
| `resolved ccr … is not the bundled claude-code-router /` | The `ccr` in use is stale or is the npm `@musistudio` doppelgänger — it lacks the `ccr restart` subcommand, so `restart` is read as a profile name. → `claude-ccr-build`, and `npm rm -g @musistudio/claude-code-router` if that is where the other one came from. |

| Symptom | Cause / fix |
|---|---|
| `Profile not found` | |
| `$NAME_API_KEY is empty` | The key isn't set in your keys file. |
| `offline and no models.dev cache` | Run `claude-providers sync` once online to populate the cache. |
| A key didn't become an alias | It's `unmapped` (no models.dev match) or classified `vcs/infra`. Map it with `add`, or check `scripts/providers/evidence/mapping-report.md`. |
| Alias missing after sync | `source ~/.local/share/claude-multi-account/aliases.sh` or open a new shell. |
| 400 "exceeded model token limit" | The catalog row for the model is self-inconsistent (`limit.output >= limit.context`). `derive_limits()` in `scripts/providers_resolve.py` always **carves** the output cap out of the context instead of trusting the published one, so such a row resolves to the same cap an absent output would; re-run `claude-providers sync` to rewrite the alias `.env`. A model the catalog has never heard of (an operator pin) no longer emits *no* limits either — an unknown context resolves to a conservative measured fallback (the catalog p10, narrowed by the provider's own p10, capped by that provider's widest window and floored so it stays usable), so both guards always exist. A `:free` row whose output budget exceeds its paid sibling's is only adjudicated at all when its `output < context` (where they are equal the row is just the common output-copied-from-context mislabel, which the carve already fixes), and its context is only lowered when at least three *distinct* providers **other than the accused one**, publishing the same model under a compatible vendor prefix, put it lower — then only to the lower median of what they publish. A lone suspicious row is left alone, because a wrongly-shrunk window is its own dead alias. Note the deliberate blind spot in the other direction: a genuinely throttled `:free` tier is corroborated at the *paid* value and left alone, since every peer record describes the paid tier. |
| `# FAIL: route-mismatch` in a `--deep` evidence file | The verification turn was served by a different backend than the alias under test (see §7). Re-run that leg on its own rather than after another router alias. |

| Symptom | Cause / fix |
|---|---|
| `# FAIL: route-mismatch-background` | Same, for the `.Router.background` entry: background sub-requests of that turn went to another backend (see §7). |
| `# FAIL: route-unproven` | The route was written but nothing proves the live gateway loaded it — the `ccr restart` was refused or never happened. Most often an authenticated gateway restarted without `CCR_API_KEYS` visible; re-run the leg with the gateway's keys in the environment. |
| Alias is `verified`, config looks right, **every launch dies as an apparent timeout/hang** | The vendor-prefix routing class fixed in **v1.26.7** — see §9.1. Confirm with `tail ~/.claude-code-router/<alias-id>/service.log`: a burst of 503s at **1-4 ms each** means the *local* gateway rejected the route before any network call, so no `--timeout` value helps. Rebuild the router (`claude-ccr-build`) if you are on an older build. |
| `# FAIL: stream-aborted` in a `--deep` evidence file | The bound expired *and* the CLI wrote a result with `terminal_reason=aborted_streaming` (the marker carries `output_tokens`, `duration_ms`, `bound`). Explicitly **not** a trust/overwrite dialog. Where it died is not determined by the leg — read the alias's `service.log` as above. |
| 400 naming … `of tool input` … and a total over the window | Your enabled-plugin surface is larger than `CMA_TOOL_TOKEN_BUDGET` (default 80000), which **inflates** the derived output cap. See §9.2 — raise the budget, cut plugins, or move to a wider-context provider. |

## 9.1 Vendor-prefixed model ids (fixed in v1.26.7)

A route is `provider,model` on disk, but the router also accepts a **client-supplied** selector in the request's `model` field, and it recognises *two* separators — comma and slash, leftmost wins (`claude-code-router:internal/router/selector.go`).

Once `lib.sh` began exporting `ANTHROPIC_MODEL` for the **router** transport too, Claude Code put the raw catalog id in the request body — `deepseek-ai/DeepSeek-V3.2-TEE`, `nvidia/nemotron-…`, `Qwen/…` — the router read the **vendor** as a provider name, found none configured in the per-alias config, and returned an unknown-provider error instead of falling through to `Router.default`. The gateway answered **503 in 1-4 ms — before any network call** — and retried with backoff until the launch bound expired. **16 of 24 verified aliases** were affected.

What made it hard to see: the on-disk route was correct the whole time (`"kilo,nvidia/nemotron-…"` parses fine, its comma comes first), so the config looked healthy while every launch died. And layers 1-3 of verification curl the provider's own `base_url` directly and never touch the ccr route path, so they stayed green throughout. **Only the layer-4 live launch exercises the real chain.**

The fix decides the slash form from **evidence, not from the separator**:

- if some configured provider actually serves the whole string, it is a catalog id and routes normally;
- if a vendor prefix *collides* with a real provider name (provider `nvidia` serving `nvidia/nemotron-…`), the whole id wins — the split half is absent from `Models` while the full string is present;
- if nobody serves it, the unknown-provider error still stands, so `Ghost/whatever` cannot silently reach `Router.default`.

**Comma selectors are unchanged and still fail loudly.** A comma is this project's own route syntax and appears in no model id, so an unknown provider really is a caller error; silently redirecting it to `Router.default` would bill an upstream you never chose.

## 9.2 The launch token guards and your plugin surface

The launch wrapper exports two co-derived guards for every alias with a known context: `CLAUDE_CODE_MAX_OUTPUT_TOKENS` (output cap) and `CLAUDE_CODE_AUTO_COMPACT_WINDOW` (compaction trigger). The window must reserve for the **tool payload** as well as text and output, because the endpoint counts all three against the same window:

```
window = context - output - CMA_TOOL_TOKEN_BUDGET        # budget
default 80000
```

and when that window would fall below `CMA_MIN_COMPACT_WINDOW` (default 120000), the wrapper buys the window back by lowering the output cap instead:

```
CLAUDE_CODE_MAX_OUTPUT_TOKENS = context - CMA_MIN_COMPACT_WINDOW -
CMA_TOOL_TOKEN_BUDGET
```

**Be exact about which direction is dangerous.** An *under*estimate does not merely compact earlier — because the output cap is derived by subtracting the budget, too small a budget **inflates** the cap and the launch dies with a 400 naming the real numbers. Measured live on a host with **227 enabled plugins**, real tool input **158112** against the 80000 default gave:

`46536 text + 158112 tools + 62144 output = 266792 against a 262144 window`

Critically, `158112 + 120000 > 262144`: on such a host **no output cap makes a 262144-context provider fit**, because the tool payload plus the minimum compaction window already exceed the whole context. The only remedies are reducing the enabled-plugin surface or moving to a wider-context provider.

**Override per host** with `CMA_TOOL_TOKEN_BUDGET`, either as a line in the provider's resolved `.env` or in the shell. To measure your own tool input, read it out of the 400 itself — the provider's message names it, e.g. `you requested about 371727 tokens (199347 of text input, 70236 of tool input, 102144 in the output).`

**Verify the context before calling an alias "wide enough."** Read the resolved env file rather than any table in this guide, since the value is derived per selected model at sync time:

```
grep CMA_PROVIDER_CONTEXT_LIMIT ~/.local/share/claude-multi-
        account/providers/<alias>.env
```

A context too small to reserve the budget at all yields no window: that provider cannot serve Claude Code's tool prefix regardless, and the launch fails loudly rather than being lied to by a guard that cannot hold.

# 10. Remote distributed deployment & heavy testing

For heavy testing that depends on **real production services**, the toolkit can boot the full LLMsVerifier System on a remote host (registered in `config/containers/nezha.env`) via the `containers` submodule + podman, and run real per-provider verification through it.

- **Deployment guide + boot procedure:** `config/containers/llmsverifier/README.md` (services, ports, the exact `podman-compose` boot steps, and every root-caused deployment fix).
- **What runs:** the `llm-verifier` API + an observability tier (prometheus + grafana with an auto-provisioned datasource & dashboard + node-exporter).
- **Heavy testing:** the bundled `model-verification` tool runs a real "Do you see my code?" check against each provider's live API:

```
ssh <user>@<host> 'podman run --rm --env-file ~/helix-system/
        llmsverifier/.env \
  llm-verifier-mv:nezha --provider deepseek --model deepseek-
        chat --verbose'
```

- **Evidence:** every verification + deployment claim is captured under `docs/qa/20260616-infra/` (proofs, sweeps, security, observability).

# 11. Command quick-reference

```
# Provider aliases
claude-providers sync                  # discover keys -> create/
        refresh aliases
claude-providers sync --no-verify      # skip verification (faster)
claude-providers list                  # alias | provider |
        transport | models
claude-providers show <id>             # one provider's resolved
        env
claude-providers remove <id>           # remove alias + env, back
        up config dir
claude-providers add --from-key VAR --id PROVIDER   # register
        mapping + sync

# Launch a provider session (after: source the alias file or open
        a new shell)
deepseek                               # router provider -> claude
        via ccr
<router-provider>                      # routed provider -> claude
        via ccr
deepseek -p "your prompt"              # non-interactive print mode

# Kimi Code CLI (v1.27.0)
kimi-deepseek                          # Kimi Code on the SAME
        backend as deepseek
kimi-deepseek -p "your prompt"
        # Kimi Code, non-interactive print mode
kimi-providers list                    # both alias kinds, agent
        (claude/kimi) column
kimi-providers migrate-names
        # run the one-time kimi-* -> kc-* rename

# Accounts (unchanged, still works)
claude-add-account --alias claudeN     # add a Claude account
claude-list-accounts                   # status of all accounts
claude-unify                           # re-merge shared state

# Docs
claude-export-docs                     # regenerate HTML/PDF/DOCX

# Release gate (run before tagging a release)
claude-release-gate                    # sandbox suite + LIVE real-
        alias smoke
claude-release-gate --verify-providers  # also run the full
        LLMsVerifier scan
```

# 12. Individual provider notes

## Kimi Code — OAuth subscription (kc-for-coding, kc-k3, kc-k2p7, kc-for-coding-highspeed)

If you are signed into **Kimi Code** (the `kimi` CLI), every model your subscription serves becomes an alias automatically — **Kimi 3** (`kc-k3`, 1M context, reasoning), **Kimi 2.7** (`kc-k2p7`), `kc-for-coding-highspeed`, and the account default `kc-for-coding`. These open **Claude Code** on a **Kimi-native** backend; the `kimi-<id>` twins open **Kimi Code** (see §4.2). The aliases were renamed from the legacy `kimi-*` names once, at `sync`, by `migrate-names` (see §4.3). No API key is required; the OAuth session in `~/.kimi-code/credentials/kimi-code.json` is used.

- **Discovery**: `claude-providers sync` queries `GET /coding/v1/models` with your OAuth token and emits one alias per served model (unioned with the models.dev catalog, because the listing under-reports). Each alias is verified with the strict sentinel + tool-calling + semantic pipeline.
- **Token freshness**: the OAuth token lives ~15 minutes. At every launch the wrapper reads the **live** credentials file (with expiry), refreshes via `kimi -p hi` when expired, and only then falls back to the sync-time snapshot — launches never die of a stale token.
- **kimi_proxy**: k3 enforces a "moonshot-flavored" JSON schema for tools (every `$ref` must start with `#/$defs/`). Claude Code's tool schemas would 400 without it, so all */Claude-Code-on-kimi launches route through a local normalizing proxy (`scripts/proxy/kimi_proxy.py`, installed by `install.sh`).
- **Precedence**: an OAuth subscription wins over `KIMI_API_KEY` / `ApiKey_Kimi` records for `kc-for-coding`; the API keys remain the fallback on hosts without the OAuth session.

## z.ai Coding Plan (zai-coding-plan)

The z.ai Coding Max-Yearly Plan provides access to Zhipu AI's flagship GLM models through a dedicated coding-optimized API endpoint. This is separate from the base z.ai plan — the Coding Plan endpoint uses `api.z.ai/api/coding/paas/v4` and includes **Free access** to `glm-5.2` (the flagship with 1M context and reasoning) and `glm-4.7` (the fast model with 204k context, reasoning, and tool_call support), among others.

### How the alias works

The key variable `ZAI_API_KEY` in your keys file is mapped to the `zai-coding-plan` provider ID via `scripts/providers/key-aliases.json`. An override in `scripts/providers/overrides.json` pins the strong model to `glm-5.2` and the fast model to `glm-4.7`, which are the optimal choices for coding workloads on this plan.

Transport is **router** — the alias launches through `claude-code-router` (`ccr default-claude-code`), which translates the Anthropic protocol to the OpenAI-compatible z.ai API.

### Models available on the Coding Plan

All models are free on the Coding Max-Yearly Plan:

| Model | Context | Reasoning | Tool Call | Notes |
|---|---|---|---|---|
| **glm-5.2** | 1M tokens | Yes | Yes | Flagship — newest, most capable |
| glm-5.1 | — | Yes | Yes | Intermediate |
| glm-5-turbo | — | — | — | Turbo variant of glm-5 |
| glm-5 | — | — | — | Base generation 5 |
| **glm-4.7** | 204k tokens | Yes | Yes | Fast model — reasoning + tool_call |
| glm-4.6 | — | — | — | Mid-range |
| glm-4.5-air | — | — | — | Lightweight |
| glm-4.5 | — | — | — | Base model |

**Note on glm-5.2 rate limits:** The Coding Max-Yearly Plan enforces a Fair Usage Policy on the flagship model: rapid-fire requests (e.g., repeated programmatic calls at short intervals) may trigger a temporary rate limit (error code `1313`). Normal interactive use via `zai-coding-plan` at human pace is unaffected.

### Setup

Everything is already configured on this host, but for a fresh install:

1. Ensure `ZAI_API_KEY` is exported in your keys file (`~/api_keys.sh`).
2. Run `claude-providers sync` to discover the key and create the alias.
3. `source ~/.local/share/claude-multi-account/aliases.sh` (or open a new shell).

4. Run `zai-coding-plan` to start a Claude Code session on glm-5.2.

No additional manual steps are needed — the key alias, model override, and endpoint are all defined in the provider config files.

### Usage example

```
zai-coding-plan                    # launch Claude Code on glm-5.2
zai-coding-plan -p "explain this function"   # non-interactive
        print mode
```

## Xiaomi MiMo (xiaomi)

Xiaomi MiMo is Xiaomi's LLM platform. MiMo does expose a genuine **Anthropic-native endpoint** (`https://api.xiaomimimo.com/anthropic`, `POST /anthropic/v1/messages`), and the `xiaomi` alias originally used it — but as of **v1.19.0** the alias is pinned to **router transport** on MiMo's OpenAI-compatible `/v1` endpoint instead, along with every other hosted provider. Both paths were verified working; the uniform one is far easier to debug. Launches therefore go through `claude-code-router` (`ccr`).

### How the alias works

The key variable `XIAOMI_MIMO_API_KEY` in your keys file does **not** match the models.dev `xiaomi` provider's documented env name (`XIAOMI_API_KEY`), so `scripts/providers/key-aliases.json` maps `XIAOMI_MIMO_API_KEY → xiaomi`. An override in `scripts/providers/overrides.json` pins:

- **transport** `router`
- **base_url** `https://api.xiaomimimo.com/v1` (the OpenAI-compatible endpoint)
- **strong_model** `mimo-v2.5-pro`
- **fast_model** `mimo-v2.5`

The pinning is deliberate: models.dev lists a `mimo-v2.5-pro-ultraspeed` id that the **live API does not serve**, so the override guarantees only live-served model ids are used.

The resolved alias carries a **1048576-token** context (verify with `grep CMA_PROVIDER_CONTEXT_LIMIT ~/.local/share/claude-multi-account/providers/xiaomi.env`), which makes it one of the few aliases that comfortably absorbs a large enabled-plugin tool payload — see §9.2.

### Models

MiMo's text-generation models (all support tool/function calling and reasoning; verified live 2026-06-19):

| Model | Context | Reasoning | Tool Call | Notes |
|---|---|---|---|---|
| **mimo-v2.5-pro** | 1M tokens | Yes | Yes | Flagship — strongest (alias strong) |
| **mimo-v2.5** | 1M tokens | Yes | Yes | Omni / multimodal (image, audio, video) — **alias fast** |
| mimo-v2-pro | — | Yes | Yes | Legacy v2 Pro |
| mimo-v2-omni | 256k tokens | Yes | Yes | Omni / multimodal (v2) |
| mimo-v2-flash | 256k tokens | Yes | Yes | Cheapest tier (no longer the pinned fast model) |

Non-text models exist (`mimo-v2.5-asr`, `mimo-v2.5-tts*`, `mimo-v2-tts`) but are speech-recognition / text-to-speech and cannot serve chat or code, so they are intentionally not wired as aliases.

### Verified live

- `GET /v1/models` → HTTP 200, 10 models served (the 5 text models above + 5 ASR/TTS).
- `POST /anthropic/v1/messages` with `Authorization: Bearer` → HTTP 200, native Anthropic response (`type:"message"`, `content:[{type:"text"},{type:"thinking"}]`) for both `mimo-v2.5-pro` and `mimo-v2-flash`. *(The Anthropic-native endpoint works; the alias no longer uses it — see the transport note above.)*
- `POST /v1/chat/completions` with `tools[]` → `finish_reason:"tool_calls"`, valid tool-call array, `reasoning_content` present (tool calling works).
- Streaming (`stream:true`) → SSE `chat.completion.chunk` deltas.
- Rate limits: 100 RPM / 10M TPM per account for text models (per-account, not per-key); expect `429` under load, retry with backoff.

### Setup

Everything is already configured, but for a fresh install:

1. Ensure `XIAOMI_MIMO_API_KEY` is exported in your keys file (`~/api_keys.sh`).
2. Run `claude-providers sync` to discover the key and create the alias.
3. `source ~/.local/share/claude-multi-account/aliases.sh` (or open a new shell).
4. Run `xiaomi` to start a Claude Code session on `mimo-v2.5-pro`.

### Usage example

```
xiaomi                                # launch Claude Code on mimo-
        v2.5-pro
xiaomi -p "explain this function"  # non-interactive print mode
```

## OpenCode Zen (provider id `opencode`, alias `opencode-zen`)

OpenCode Zen is OpenCode's curated AI gateway — a tested and verified list of models from multiple providers, accessed through a single API key. Zen includes **21 free models** (all $0 cost, all support tool calling and reasoning), plus 49 paid models from OpenAI, Anthropic, Google, DeepSeek, and others.

The free models are available for a limited time while OpenCode collects feedback. One of them — **Big Pickle** — is a stealth model (its true identity is not disclosed; observed as deepseek-v4-flash behind the scenes).

### How the alias works

The key variable `ZEN_API_KEY` (or `ApiKey_Opencode_Zen`) in your keys file is mapped to the `opencode` provider ID via `scripts/providers/key-aliases.json`. An override in `scripts/providers/overrides.json` pins:

- **alias** `opencode-zen` — so the launcher is `opencode-zen`, not the bare provider id. Hosts synced before that pin may still carry an `opencode` alias as well; `claude-providers list` is authoritative for what exists on yours.
- **strong_model** `big-pickle` — free stealth model, 200K context, reasoning + tool_call
- **fast_model** `deepseek-v4-flash-free` — free, 200K context, reasoning + tool_call

Those two are the *pins*. The per-model multi-alias phase (§5) additionally generates `opencode-zen1`, `opencode-zen2`, … over the free-tier models it verifies, each with its own strong/fast pair, so the resolved model for a given alias is whatever its `.env` says — check with `claude-providers show <id>`.

Transport is **router** — the alias launches through `claude-code-router` (`ccr default-claude-code`), which translates the Anthropic protocol to the OpenAI-compatible Zen API (`https://opencode.ai/zen/v1/chat/completions`).

No transport or base_url override is needed — the models.dev catalog already has the correct values (`@ai-sdk/openai-compatible` → router, `https://opencode.ai/zen/v1`).

### Free models available on Zen

All free models support tool calling and reasoning (verified live 2026-06-20):

| Model | Context | Notes |
|---|---|---|
| **big-pickle** | 200K | Stealth model — alias default (strong) |
| **deepseek-v4-flash-free** | 200K | Alias fast model |
| mimo-v2.5-free | 200K | Xiaomi MiMo |
| mimo-v2-pro-free | 1M | Xiaomi MiMo Pro |
| mimo-v2-flash-free | 262K | Xiaomi MiMo Flash |
| mimo-v2-omni-free | 262K | Xiaomi MiMo Omni |
| nemotron-3-ultra-free | 1M | NVIDIA Nemotron |
| nemotron-3-super-free | 204K | NVIDIA Nemotron Super |
| north-mini-code-free | 256K | North Mini Code |
| glm-4.7-free | 204K | Zhipu GLM |
| glm-5-free | 204K | Zhipu GLM 5 |
| grok-code | 256K | xAI Grok Code |
| kimi-k2.5-free | 262K | Moonshot Kimi |
| minimax-m2.1-free | 204K | MiniMax |
| minimax-m2.5-free | 204K | MiniMax |
| minimax-m3-free | 200K | MiniMax |
| qwen3.6-plus-free | 262K | Alibaba Qwen |
| ring-2.6-1t-free | 262K | Ring |
| hy3-preview-free | 256K | HY3 Preview |
| ling-2.6-flash-free | 262K | Ling (no reasoning) |
| trinity-large-preview-free | 131K | Trinity (no reasoning) |

Paid models start at $0.05/1M tokens (GPT-5 Nano) and go up to $30/1M (GPT-5.5 Pro). Claude models are available from $1/1M (Haiku 4.5) to $15/1M (Opus 4.1).

### Verified live

- `GET /v1/models` → HTTP 200 (model list returned).
- `POST /v1/chat/completions` with `big-pickle` → HTTP 200, correct text response, cost=$0, reasoning_content present. Stealth alias observed as deepseek-v4-flash.
- `POST /v1/chat/completions` with `deepseek-v4-flash-free` → HTTP 200, correct text, cost=$0.

- Additional free models (`mimo-v2.5-free`, `nemotron-3-ultra-free`, `north-mini-code-free`) → all HTTP 200, all cost=$0.
- Streaming (`stream:true`) → SSE `chat.completion.chunk` deltas.

### Setup

Everything is already configured, but for a fresh install:

1. Ensure `ZEN_API_KEY` (or `ApiKey_Opencode_Zen`) is exported in your keys file (`~/api_keys.sh`).
2. Run `claude-providers sync` to discover the key and create the alias.
3. `source ~/.local/share/claude-multi-account/aliases.sh` (or open a new shell).
4. Run `opencode-zen` to start a Claude Code session on `big-pickle` (via ccr).

### Usage example

```
opencode-zen                              # launch Claude Code on
        big-pickle (via ccr)
opencode-zen -p "explain this function" # non-interactive print
        mode
claude-providers list                     # which opencode-zenN
        aliases exist here
```

### Notes

- The alias uses **router transport** (ccr) because Zen's free models use OpenAI-compatible format, not Anthropic native format.
- Big Pickle is a stealth model — the actual model served may vary. This is by design per OpenCode's documentation.
- To use paid Claude models on Zen instead of free models, override `strong_model` and `fast_model` in `scripts/providers/overrides.json` (e.g. `claude-haiku-4.5` and `claude-sonnet-4-6`), and set `transport` to `native` with `base_url` to `https://opencode.ai/zen/v1/messages`.

## Chutes (chutes)

Chutes is a pay-per-use AI inference platform with **TEE (Trusted Execution Environment)** models — all models run in secure enclaves for data privacy. The API is OpenAI-compatible at `https://llm.chutes.ai/v1`.

### How the alias works

The key variable `CHUTES_API_KEY` in your keys file maps to the `chutes` provider ID. An override in `scripts/providers/overrides.json` pins:

- **strong_model** `zai-org/GLM-5.2-TEE` — newest GLM model, 202K context
- **fast_model** `Qwen/Qwen3.6-27B-TEE` — fast Qwen model, 262K context
- **context_limit** `262144` and **max_output** `65536`

Transport is **router** — the alias launches through `claude-code-router` (`ccr default-claude-code`), which translates the Anthropic protocol to the OpenAI-compatible Chutes API.

Two things follow from those pins and are worth knowing before you debug a Chutes launch:

- Every Chutes model id is **vendor-prefixed** (`zai-org/…`, `Qwen/…`, `deepseek-ai/…`, `moonshotai/…`) — the exact shape that made 16 of 24 verified aliases unroutable before **v1.26.7**. If launches die as apparent timeouts, read §9.1 and confirm your router is current (`claude-ccr-build`).
- The **262144** context is the window §9.2 works through: on a host with a large enabled-plugin surface the tool payload alone can make it unusable regardless of the output cap.

### Models available on Chutes

All models are TEE-enabled (Trusted Execution Environment):

| Model | Context | Notes |
|---|---|---|
| **zai-org/GLM-5.2-TEE** | 202K | Newest GLM — alias default (strong) |
| zai-org/GLM-5.1-TEE | 202K | GLM 5.1 |
| zai-org/GLM-5-TEE | 202K | GLM 5 |
| Qwen/Qwen3.5-397B-A17B-TEE | 262K | Largest Qwen model |
| **Qwen/Qwen3.6-27B-TEE** | 262K | Alias fast model |
| Qwen/Qwen3-32B-TEE | 40K | Qwen 32B |
| Qwen/Qwen3-235B-A22B-Thinking-2507-TEE | 262K | Thinking/reasoning model |
| deepseek-ai/DeepSeek-V3.2-TEE | 131K | DeepSeek V3.2 |
| moonshotai/Kimi-K2.5-TEE | 262K | Kimi K2.5 |
| moonshotai/Kimi-K2.6-TEE | 262K | Kimi K2.6 |
| MiniMaxAI/MiniMax-M2.5-TEE | 196K | MiniMax M2.5 |

| Model | Context | Notes |
|---|---|---|
| google/gemma-4-31B-turbo-TEE | 131K | Google Gemma 4 |
| unsloth/Mistral-Nemo-Instruct-2407-TEE | 131K | Mistral Nemo |

**Important:** Chutes is **pay-per-use** — the account must be funded to use models. Add balance at https://chutes.ai. The API key and configuration are correct, but requests will return "Quota exceeded" if the account balance is $0.

### Setup

1. Ensure `CHUTES_API_KEY` is exported in your keys file (`~/api_keys.sh`).
2. Fund your Chutes account at https://chutes.ai.
3. Run `claude-providers sync` to discover the key and create the alias.
4. `source ~/.local/share/claude-multi-account/aliases.sh` (or open a new shell).
5. Run `chutes` to start a Claude Code session on `zai-org/GLM-5.2-TEE`.

### Usage example

```
chutes                                  # launch Claude Code on
        GLM-5.2-TEE (via ccr)
chutes -p "explain this function"   # non-interactive print mode
```

### Verified

- API endpoint `https://llm.chutes.ai/v1/chat/completions` responds correctly
- Authentication with `Authorization: Bearer cpk_...` works
- All 13 TEE models are accessible (require funded account)
- OpenAI-compatible format confirmed

## Poe (poe)

Poe is a universal AI platform with **382 models** from all major providers — chat, code, image generation, video generation, TTS, and STT. The API is OpenAI-compatible at `https://api.poe.com/v1`.

### How the alias works

The key variable `POE_API_KEY` (or `ApiKey_Poe`) in your keys file maps to the `poe` provider ID. Transport is **router** — the alias launches through `claude-code-router` (`ccr default-claude-code`).

### Aliases

Only one Poe alias is **pinned**; `scripts/providers/overrides.json` sets:

| Alias | Strong Model | Fast Model | Focus |
|---|---|---|---|
| **poe** | claude-sonnet-4.6 | gpt-5.4-mini | Primary — best Claude |

Any additional `poe2`, `poe3`, … aliases are **generated dynamically** by the per-model multi-alias phase (§5), not pinned here — and since that phase probes **free-tier models only** by default, a provider whose catalogue is entirely paid produces no extra aliases unless you opt in with `--include-paid`. Run `claude-providers list` to see what actually exists on your host rather than assuming the numbered aliases are present.

### Model categories (382 total)

| Category | Count | Examples |
|---|---|---|
| **Chat/ Reasoning** | 130 | claude-opus-4.8, gpt-5.5, grok-4, deepseek-v4-pro, gemini-3.1-pro, qwen3.7-max |
| **Code** | 16 | claude-code, gpt-5.3-codex, qwen3-coder-next, kimi-k2.7-code, seed-2.0-code |
| **Image Generation** | 40 | flux-2-pro, imagen-4, stable-diffusion3.5, qwen-image-2, grok-imagine-image |
| **Video Generation** | 17 | sora-2-pro, veo-3.1, kling-3.0, runway-gen-4.5, pixverse-v5.6 |
| **TTS/Voice** | 12 | elevenlabs-v3, gemini-3.1-flash-tts, cartesia-ink-whisper, orpheus-tts |
| **STT/Speech** | 1 | whisper-v3-large-t |
| **Other** | 166 | perplexity-search, exa-research, hailuo, minimax-m3, glm-5.2, and more |

### Supported capabilities

- **Chat completions** — OpenAI-compatible `/v1/chat/completions`
- **Tool calling** — verified on claude-sonnet-4.6, gpt-5.4-mini, deepseek-v4-pro-e, grok-4
- **Streaming** — standard SSE streaming supported
- **Reasoning** — reasoning_content field in responses
- **Image input** — vision models support image URLs
- **Video generation** — via separate video endpoint

- **TTS/STT** — text-to-speech and speech-to-text models

### Setup

1. Ensure `POE_API_KEY` is exported in your keys file (`~/api_keys.sh`).
2. Run `claude-providers sync` to discover the key and create the alias.
3. `source ~/.local/share/claude-multi-account/aliases.sh` (or open a new shell).
4. Run `poe` to start a Claude Code session on `claude-sonnet-4.6` (via ccr).

### Usage example

```
poe                                  # launch Claude Code on
        claude-sonnet-4.6 (via ccr)
poe -p "explain this function"       # non-interactive print mode
claude-providers list                # check which poeN aliases
        exist on this host
```

### Verified

- API endpoint `https://api.poe.com/v1/chat/completions` responds correctly
- Authentication with `Authorization: Bearer sk-poe-...` works
- Tool calling verified on multiple models
- 382 models accessible across all categories
- OpenAI-compatible format confirmed

## HelixAgent (helixagent) — local single-GPU model

`helixagent` points Claude Code at a **local** HelixLLM backend — a podman container serving Qwen3-Coder-30B on one GPU (an RTX 5090) at `http://127.0.0.1:7061/v1` (measured 2026-09-03; the earlier `:18434` pin named the llama.cpp coder container — a different service, and down when measured) — instead of a hosted API. Transport is **router**: launches go through `claude-code-router` plus the bundled Go `cma-proxy`, which recovers the model's Hermes/Qwen `<function=…>` tool calls into structured `tool_calls` so Claude Code's tools actually engage. The alias is pinned in `scripts/providers/helixagent.json` to a **229,376-token** context window and an **8,192-token** output cap.

Two things are specific to a local, single-GPU backend and worth knowing before you launch it.

### The backend must be in "claude mode" (one big slot), not "coder mode"

The HelixLLM backend is shared — on the **same single GPU** — with HelixCode, and the two configurations are **mutually exclusive**:

| Mode | llama.cpp config | Slot layout | For |
|---|---|---|---|
| **claude mode** | `-c 229376 --parallel 1` | one 229,376-token slot | the `helixagent` alias (Claude Code) |
| **coder mode** | `-c 24576 --parallel 8` | eight 3,072-token slots | HelixCode's concurrent sub-requests |

llama.cpp splits `-c` across the parallel slots, so **coder mode gives each request only ~3,072 tokens** — a Claude Code session (system prompt + tool schemas) is far larger, and every launch against coder mode returns **HTTP 400**. `helixagent` needs the single large slot of claude mode.

Because coder mode is the **default** operational state (HelixCode is the common case), `helixagent` is honestly demoted to **`unverified`** and the launch gate refuses it until you flip the backend to claude mode:

```
# The mode-switch script lives in the companion helix_code repo — NOT this toolkit:
helix_code/scripts/helixllm-mode.sh claude        # one 229,376-token slot
# then re-verify + launch:
claude-providers verify helixagent --deep      # passes once the backend is in claude mode
helixagent
```

Switch back with `helixllm-mode.sh coder` when you need HelixCode again. The two modes cannot run at once on the shared GPU.

### Auto-start is opt-in — `CMA_HELIX_AUTOSTART`

Launching `helixagent` does **not** start HelixLLM for you by default. Opt in per launch (or export it) with any of the truthy values `1`, `true`, `yes`, `on` — matched case-insensitively; anything else, unset included, means off:

```
CMA_HELIX_AUTOSTART=1 helixagent
```

The default is off because booting HelixLLM is a **machine-wide** side effect, not a private one: it claims the single GPU, and starting it in claude mode **evicts whatever HelixCode had running in coder mode**. A provider alias must not do that to a host merely because someone launched it.

Turning it off costs you no information. When HelixLLM is not answering, the pre-flight still says so and still prints the exact command:

```
helixagent: HelixLLM is not answering on http://127.0.0.1:7061 and
auto-start is OFF by default.
  Start it yourself (it claims the GPU and evicts HelixCode's
coder mode):
    helix_code/scripts/helixllm-mode.sh claude
  Or opt in for this launch:
    CMA_HELIX_AUTOSTART=1 <alias>
```

With the opt-in set, the wrapper searches the usual Helix Code locations (and `$HELIX_CODE_DIR`) for `helixllm-mode.sh`, runs it in claude mode, then polls `/health` for up to 120 s — reporting a timeout with `podman logs helixllm-coder` and `<script> status` as next steps rather than proceeding silently. Auto-start only covers the *not running* case: a backend that is up but sitting in coder mode (`context_limit=24576`) gets the separate coder-mode warning above, and is never switched out from under HelixCode.

### Minimal-launch (`--bare`) trim mode — `CMA_PROVIDER_TRIM='bare'`

A local 229,376-token window is small next to a hosted model's, and a normal launch would overflow it before the first turn: on a plugin-heavy host, auto-resumed session history (~330k tokens) plus the fixed hook/plugin/MCP/CLAUDE.md tool schemas (~110k tokens) together dwarf the whole window. So `helixagent`'s resolved env file carries `CMA_PROVIDER_TRIM='bare'`, which makes **every conversation launch minimal and fresh**:

- prepends **`--bare`**, dropping the hook/plugin/MCP/CLAUDE.md surface;
- **skips both automatic history seams** — the conversation-args auto-`--resume` injection *and* the interactive (zero-args) stored session-flags injection — so no synced session history rides along;
- so each launch is a **fresh session that fits the local window**.

What trim deliberately does **not** touch:

- **Explicit** session selectors you pass yourself (`--resume <id>`, `--session-id <id>`, `--continue`, `--fork-session`, `-c`) are honored **verbatim** — trim only suppresses the *automatic* resume, never your own.

- **Non-conversation subcommands** (`agents`, `mcp`, `export`, `doctor`, `config`, `plugin`, `setup`, …) get neither `--bare` nor a resume.
- **Untrimmed providers are byte-identical to before.** Trim is opt-in per provider via the `.env` line, wired today only for `helixagent` but reusable for any local-model provider that needs a fresh minimal session each launch.

## llmctl (local, multi-profile) — one alias per RUNNING model server

llmctl is a sibling local-LLM orchestrator — a separate project, not part of this toolkit — that probes your hardware, downloads GGUF models with cryptographic verification, builds llama.cpp/colibri from pinned source, and runs multiple models concurrently as systemd user services (Linux) or launchd agents (macOS). Each catalog **profile** (`fast`, `small`, `vision`, `vision-pro`, `coder`, `moe-fast`, `colibri-qwen36`, …) is its **own independent** llama.cpp/colibri server process on a fixed, catalog-assigned port, serving an OpenAI-compatible `/v1`.

### How the alias works

`claude-providers sync` asks `llmctl plan --json` for the current catalog (profile names + ports — never hardcoded here, see the detector's own header comment in `scripts/claude-providers.sh`) and live-probes `/v1/models` on every profile's port. A profile that answers with a genuine OpenAI-shaped listing right now gets its own alias, `llmctl-<profile>` (e.g. `llmctl-fast`, `llmctl-vision`); a profile that is not currently running gets **no** alias — this is the one local provider in this toolkit that can register **zero, one, or many** aliases from a single sync, because llmctl supports running several model servers side by side (co-residency). Transport is **router** (every profile's endpoint is OpenAI-compatible, so launches go through `claude-code-router`), and no API key is required — llmctl's servers bind to `127.0.0.1` unauthenticated by default.

Strong/fast model and context window are **never pinned** — they are read live from each running profile's own `/v1/models` response (the model id llama.cpp/colibri reports, and `meta.n_ctx` when the backend publishes one, falling back to the catalog's own planned `ctx` for that profile). A profile answering with a non-JSON or non-OpenAI-shaped body — e.g. an unrelated service that happens to already own that port — is correctly treated as "not running", never mis-registered against the wrong backend.

### Setup

1. Set up llmctl in its own repository and start one or more model profiles:

```
cd /path/to/llmctl
./bin/llmctl setup                # doctor + build engines + hardware plan
./bin/llmctl models download fast # verified download of a profile
./bin/llmctl start fast           # serves 127.0.0.1:<fast's port>/v1
```

2. Back in this toolkit, run `claude-providers sync` — every currently-running profile is discovered and registered automatically, with zero manual per-model configuration.
3. `source ~/.local/share/claude-multi-account/aliases.sh` (or open a new shell).

### Usage example

```
llmctl-fast                       # launch Claude Code on the 'fast' profile (via ccr)
llmctl-fast -p "explain this function"
claude-providers list             # see exactly which llmctl-<profile> aliases exist right now
```

Stop a profile (`llmctl stop fast`) and re-run `claude-providers sync`; that alias's `.env` is left in place (the same "installed but not currently answering" honesty as `helixagent`/`helixcoder`) rather than deleted, and `claude-providers verify llmctl-fast` reports `unverified` until the profile is started again.

### Overrides

Every knob is env-overridable (`CMA_LLMCTL_BIN`, `CMA_LLMCTL_PINS_FILE`, `CMA_LLMCTL_KEYVAR`, `CMA_LLMCTL_TRANSPORT`, `CMA_LLMCTL_CONTEXT_LIMIT`, `CMA_LLMCTL_MAX_OUTPUT`, `CMA_LLMCTL_PLAN_TIMEOUT`, `CMA_LLMCTL_HTTP_TIMEOUT`) and the tracked pins file `scripts/providers/llmctl.json` sets the defaults (`bin`, `key_var`, `transport`, and the `context_limit`/`max_output` fallbacks used only when a running profile's own response names none). The pins file deliberately carries **no** base URLs or ports — those can only be learned by asking the real `llmctl plan --json`, never invented here (the same lesson the `helixagent`/`helixllm` pins learned the hard way, documented above: a stale hardcoded port is worse than no pin at all).

## 13. Releasing — the mandatory live pre-release gate

`claude-release-gate` is the release gate: **no release commit may be made unless it exits 0.** It exists because a green sandbox suite is not proof the real host works — v1.25.1 shipped with the entire sandbox suite green while **every** router alias on the real

host was bricked. The sandbox proves wrapper *logic*; it is structurally blind to real-host state (a PATH-shadowing `ccr` doppelgänger, a mis-configured local backend, resumed history overflowing a small window). So the gate adds a **live** layer that drives the real chain end-to-end.

```
claude-release-gate                       # sandbox suite + live
        smoke (default provider: helixagent)
claude-release-gate --provider poe        # gate through a
        different provider
claude-release-gate --skip-suite          # reuse a suite run you
        JUST completed green
claude-release-gate --verify-providers    # also run the full
        LLMsVerifier provider scan
```

The layers, fail-closed — any failure means **DO NOT RELEASE**:

1. **Sandbox suite** — `scripts/tests/run-all.sh`. `--skip-suite` skips it, only valid if you just ran it green.
2. **Live alias smoke** — regenerates the aliases from the *current* `lib.sh`, then launches the real alias with a fresh session (`… --session-id <uuid> -p "Reply with exactly: GATE-OK"`) through the real **PATH → ccr → route-apply → proxy → provider backend**, and asserts (a) the launch exited 0, (b) the served reply contained `GATE-OK`, and (c) for a router-transport provider, that the gateway's **sink-side route** (`.Router.default` in `~/.claude-code-router/<alias-id>/config.json`) actually names the provider under test. A write-then-apply route that silently served the wrong backend fails here.
3. **Provider scan** *(opt-in, `--verify-providers`)* — runs `claude-verify-providers` (the LLMsVerifier scan) over every provider/model. Slower; off by default.

The gate provider is chosen as `--provider`, else `$CMA_GATE_PROVIDER`, else `helixagent`. It **must exist and be verified** — a missing or broken gate provider is a gate **failure** (fix it, or pick another with `--provider`), never a silent skip. For `helixagent` specifically that means the HelixLLM backend must be in claude mode first (§12).

---

# 13. HelixCode / HelixLLM Providers (Native HelixCode Integration)

HelixCode includes a **native LLM provider system** (`internal/llm/`) that operates independently of the `claude-providers` toolkit. These providers are configured via `helix_code/config/config.yaml` and accessed through the HelixCode server API or CLI (`helixcode llm ...`).

## Key Differences from claude-providers

| Aspect | `claude-providers` | HelixCode Native |
|---|---|---|
| **Target** | Claude Code sessions | HelixCode server/CLI/TUI |
| **Configuration** | `~/api_keys.sh` + models.dev | `config/config.yaml` + env vars |
| **Verification** | Live sentinel + tool-calling probes | Health checks + LLMsVerifier |
| **Model Selection** | Credit-aware tier + scoring | Performance + capability + fallback |
| **Cloud Gate** | None (always tries) | W2c-1 gate (`llm.cloud.enabled`) |

## HelixCode Provider List (15+ Providers)

All providers implement the `Provider` interface (`internal/llm/provider.go`) and are registered via `ModelManager` (`internal/llm/model_manager.go`).

### Cloud Providers (Require `llm.cloud.enabled: true`)

| Provider | Type | Env Var | Endpoint |
|---|---|---|---|
| **Anthropic** | Cloud | `ANTHROPIC_API_KEY` | `https://api.anthropic.com` |
| **Google Gemini** | Cloud | `GEMINI_API_KEY` / `GOOGLE_API_KEY` | `https://generativelanguage.googlea` |
| **OpenAI** | Cloud | `OPENAI_API_KEY` | `https://api.openai.com` |
| **XAI (Grok)** | Cloud | `XAI_API_KEY` | `https://api.x.ai` |

| Provider | Type | Env Var | Endpoint |
|---|---|---|---|
| **Groq** | Cloud | `GROQ_API_KEY` | `https://api.groq.com` |
| **Mistral** | Cloud | `MISTRAL_API_KEY` | `https://api.mistral.ai` |
| **DeepSeek** | Cloud | `DEEPSEEK_API_KEY` | `https://api.deepseek.com` |
| **OpenRouter** | Cloud | `OPENROUTER_API_KEY` | `https://openrouter.ai/api/` |
| **Cohere** | Cloud | `COHERE_API_KEY` | `https://api.cohere.ai` |
| **GitHub Copilot** | Cloud | `GITHUB_TOKEN` | `https://api.githubcopilot.` |
| **Azure OpenAI** | Cloud | `AZURE_OPENAI_API_KEY` | Azure endpoint |
| **AWS Bedrock** | Cloud | `AWS_ACCESS_KEY_ID` + `AWS_SECRET_ACCESS_KEY` | AWS regional |
| **GCP Vertex AI** | Cloud | `GOOGLE_APPLICATION_CREDENTIALS` | GCP regional |
| **Qwen** | Cloud | `QWEN_API_KEY` / OAuth2 | `https://dashscope.aliyuncs` |
| **Replicate** | Cloud | `REPLICATE_API_TOKEN` | `https://api.replicate.com` |

### Local Providers (Exempt from Cloud Gate)

| Provider | Type | Requirements | Models | Notable Features |
|---|---|---|---|---|
| **Ollama** | Local | Ollama service on `localhost:11434` | Any GGUF | Easy mgmt, local-only, model library |
| **Llama.cpp** | Local | llama.cpp server/binary | GGUF models | Direct llama.cpp, HW accel (Metal/ CUDA) |
| **vLLM** | Local | vLLM OpenAI-compatible server | Any HF model | PagedAttention, high throughput |
| **LocalAI** | Local | LocalAI server | OpenAI-compat | Drop-in OpenAI replacement |
| **FastChat** | Local | FastChat controller/ worker | Vicuna, LLaMA | Conversation templates |
| **LM Studio** | Local | LM Studio local server | GGUF models | GUI + API server |
| **Jan** | Local | Jan app local server | GGUF models | Desktop app with API |
| **GPT4All** | Local | GPT4All local server | Quantized models | Consumer-friendly |
| **TabbyAPI** | Local | TabbyAPI server | exllama models | ExLLaMAv2 wrapper |
| **MLX** | Local | Apple Silicon + MLX | MLX-format models | Apple Silicon native |
| **Mistral.rs** | Local | mistral.rs binary | Any HF model | Rust impl, fast inference |
| **KoboldAI** | Local | KoboldAI server | Story/ writing models | Storytelling-focused |
| **Xiaomi MiMo** | Local/ Cloud | MiMo API key | MiMo v2.5 Pro/ Omni | 1M context, multimodal, tools |

### Special Providers

- **HelixAgent** — Embedded multi-agent workflow provider
- **Cerebras** — Wafer-scale inference for Llama models
- **Together AI** — Optimized open-model serving
- **HuggingFace** — Inference endpoints for HF models

## HelixCode Model Selection Strategy

HelixCode uses `ModelManager.SelectOptimalModel()` (`internal/llm/model_manager.go`) with:

1. **Capability Matching** — Matches task requirements (tools, vision, reasoning) to model capabilities
2. **Performance Scoring** — Latency, throughput, success rate from health checks
3. **Fallback Chains** — Automatic failover: primary → fallback → local
4. **LLMsVerifier Integration** — Real verification scores from `internal/verifier/`
5. **Health Monitoring** — Background poller checks provider health every 30s

**Configuration** (`config/config.yaml`):

```yaml
llm:
  selection:
    strategy: "performance"  # or "capability", "cost", "hybrid"
    fallback_enabled: true
    health_check_interval: 30
```

## HelixCode Verification — LLMsVerifier Integration

HelixCode includes a **built-in verification system** (`internal/verifier/`):

- **REST API Client** — Connects to LLMsVerifier service
- **Two-Tier Cache** — Memory + Redis cache with TTL
- **Circuit Breaker** — Prevents cascade failures
- **Background Poller** — Refreshes model scores periodically
- **Score Adapter** — Normalizes verifier scores to selection weights
- **Event Publisher** — Emits provider status changes

**Verification Flow:**

```
ModelManager.SelectOptimalModel()
  → VerifierAdapter.GetScores()  // cached or live
  → ScoreAdapter.Adapt()         // normalize to weights
  → CapabilityFilter()           // match task requirements
  → PerformanceRank()            // latency + success rate
  → Return optimal model
```

## HelixCode CLI Usage

```
# List configured providers with status
helixcode llm providers list
```

```
# List available models across all providers
helixcode llm models list

# Set default provider
helixcode llm provider set anthropic

# Generate with specific model
helixcode llm generate --model claude-4-sonnet "design a cache
        system"

# Chat with history
helixcode llm chat --model gemini-2.5-pro

# Run provider health checks
helixcode llm health

# Verify models via LLMsVerifier
helixcode llm verify --provider anthropic --model claude-4-sonnet
```

## HelixCode Server API

```
# List providers with health status
curl -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/
        v1/llm/providers

# List models
curl -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/
        v1/llm/models

# Generate completion
curl -X POST -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"model": "claude-4-sonnet", "messages": [{"role": "user",
        "content": "Hello"}]}' \
  http://localhost:8080/api/v1/llm/generate
```

## Provider Capabilities Matrix

| Provider | Tools | Vision | Reasoning | Streaming | Max Context | Local |
|---|---|---|---|---|---|---|
| Anthropic | ✓ | ✗ | ✓ (extended) | ✓ | 200K | ✗ |
| Gemini | ✓ | ✗ | ✗ | ✓ | 2M | ✗ |
| OpenAI | ✓ | ✗ | ✓ (o1/o3) | ✓ | 1M+ | ✗ |
| XAI | | | | | 128K | |
| Groq | | | | | 128K | |
| Mistral | | | | | 32K | |
| DeepSeek | | | ✓ (R1) | | 64K | |

| Provider | Tools | Vision | Reasoning | Streaming | Max Context | Local |
|---|---|---|---|---|---|---|
| OpenRouter | | * | * | * | Varies* | |
| Cohere | | | | | 128K | |
| Copilot | | * | * | * | 128K* | |
| Azure | | * | * | * | 128K* | |
| Bedrock | | * | * | * | 200K* | |
| VertexAI | | * | * | * | 2M* | |
| Qwen | | | | | 128K | |
| Replicate | * | * | * | * | Varies* | |
| **Ollama** | * | * | * | * | 128K* | |
| **Llama.cpp** | * | | | * | 128K* | |
| **vLLM** | * | * | * | * | Model* | |
| **LocalAI** | * | * | * | * | Model* | |
| **Xiaomi** | | | | | 1M | Both |

- = Depends on specific model deployed

## Cloud Gate (W2c-1) — Local-First Default

HelixCode enforces a **cloud gate** (`llm.cloud.enabled` in config, default `false`):

```yaml
llm:
  cloud:
    enabled: false  # Set true to allow cloud providers
```

When **disabled** (default):

- Only local providers work (Ollama, Llama.cpp, vLLM, LocalAI, etc.)
- Cloud provider construction returns `ErrCloudDisabled`
- Error message guides to local alternatives or enabling the gate

When **enabled**:

- All 15+ cloud providers available
- Requires valid API keys in environment
- Health checks run automatically

**Rationale**: Prevents accidental cloud usage/costs; operators must explicitly opt-in.

## LLMsVerifier Single Source of Truth (CONST-036)

Per CONST-036, HelixCode uses **LLMsVerifier as the sole authoritative source** for:

- Model metadata (names, IDs, context windows, capabilities)
- Provider metadata (endpoints, auth types, supported models)
- Verification status (verified, partial, failed, pending)
- Scoring data (overall scores, capability scores, tier rankings)

**No hardcoded model lists** — all model discovery flows through LLMsVerifier.

**Fallback Models** (constitutionally permitted, `internal/verifier/fallback_models.go`):

```
var FallbackModels = []FallbackModel{
  {Name: "claude-4-sonnet", Provider: "anthropic", Score: 95,
        Verified: true},
  {Name: "gemini-2.5-pro", Provider: "gemini", Score: 93,
        Verified: true},
  {Name: "gpt-4.1", Provider: "openai", Score: 92, Verified:
        true},
  {Name: "grok-3", Provider: "xai", Score: 88, Verified: true},
  {Name: "deepseek-v3", Provider: "deepseek", Score: 90,
        Verified: true},
  {Name: "mistral-large", Provider: "mistral", Score: 87,
        Verified: true},
  {Name: "llama-3.3-70b", Provider: "ollama", Score: 85,
        Verified: true},
}
```

## Anti-Bluff Guarantees (CONST-035, CONST-037, CONST-038)

- **CONST-035**: Every model displayed to users verified by LLMsVerifier within 24h
- **CONST-037**: Models >24h old show "stale" indicator, deprioritized
- **CONST-038**: Model status (available, rate-limited, cooldown, offline) reflects LLMsVerifier within 60s
- **CONST-039**: Integrates ALL providers LLMsVerifier supports (minimum: OpenAI, Anthropic, Gemini, DeepSeek, Groq, Mistral, xAI, OpenRouter, Ollama, Llama.cpp)

## Integration with claude-providers

HelixCode native providers and `claude-providers` can **coexist**:

1. **HelixCode server** uses native providers for its own LLM operations

2. **claude-providers** creates Claude Code aliases for interactive coding
3. **Shared verification** — both can use the LLMsVerifier checkout
4. **Different use cases** — HelixCode for automation/workflows, claude-providers for interactive coding

**Example workflow:**

```
# HelixCode native for automated tasks
helixcode llm generate --model claude-4-sonnet "Create REST API"

# claude-providers for interactive session
source ~/.local/share/claude-multi-account/aliases.sh
deepseek  # Opens Claude Code on DeepSeek backend
```

### Configuration Files

- **HelixCode native**: `helix_code/config/config.yaml` (or `production-config.yaml`, `minimal-config.yaml`)
- **Model aliases**: `helix_code/config/model-aliases.example.yaml` → `.helix/model-aliases.yaml`
- **claude-providers**: `~/api_keys.sh` + `scripts/providers/overrides.json` + `scripts/providers/key-aliases.json`

### Migration from Hardcoded Lists

Previously (BLUFF-002, BLUFF-004), HelixCode had hardcoded model lists in CLI. Now **fixed**:

- `cmd/cli/main.go` → Uses `ModelManager` + `VerifierAdapter` for dynamic discovery
- `internal/llm/model_manager.go` → Queries LLMsVerifier for live model catalog
- `internal/verifier/` → Full verification client with caching, circuit breaker, polling

---

## 14. Quota and limits reporting (`quota` / `limits`)

`claude-providers quota` (alias: `limits` — the two names are 100% interchangeable: same flags, same output, same exit codes) reports usage/limit windows for every configured native account and provider alias in a single pass. `kimi-providers quota` / `kimi-providers limits` report the identical data — `kimi-providers` has no separate implementation; it reaches this through its existing catch-all forward into `claude-providers.sh` (see §4.2).

```
claude-providers quota                    # fleet-wide report
claude-providers quota <alias>            # scope to one alias
claude-providers limits                   # identical to quota
kimi-providers quota                      # same report, Kimi-side
        entrypoint
kimi-providers limits <alias>             # scoped, Kimi-side
        entrypoint
```

## Flags

| Flag | Default | Meaning |
|---|---|---|
| `<alias>` (positional, optional) | none — fleet-wide | Scope the report to one alias. A name that doesn't currently exist is stated plainly (`alias "<name>" does not exist`) and the command exits `2` — never a crash, never a silent empty success. |
| `--json` | off | Machine-readable output: one JSON object (`generated_at`, `scoped_to`, `unknown_alias`, `rows[]`) instead of colored/plain text. Every field is always present in every row; `null` is a valid value but the key itself is never omitted. |
| `--fresh` | off (short-lived cache allowed) | Skip the cache and force a live re-probe for this invocation. |
| `--timeout <seconds>` | `3` | Override the per-provider probe timeout for this invocation only — never persisted. |
| `--no-color` | off (color auto-detected) | Force plain text even on a color-capable terminal. A non-terminal stdout and `NO_COLOR` (any non-empty value) already disable color automatically; this flag is the explicit, scriptable override. |

Exit codes: `0` — the command ran to completion and produced a report, true even when every row is red/limit-exceeded/not-reported (the exit code reflects whether reporting succeeded, not what it found); `1` — a genuine command-level failure (e.g. an unrecognized flag); `2` — `<alias>` was given and does not exist.

## Coloring

In text mode, each usage window is colored by its percent remaining: green at `>=30%` left, yellow `10-30%`, red `<10%`, and a distinct `limit_exceeded` state at `<=0%`. The severity is always spelled

out in words too (`[GREEN]`, `[YELLOW]`, `[RED]`), so a color-stripped read — piped to a file, `--no-color`, or `--json` — never loses the distinction.

## Honest absence vs. probe failure

These are two different outcomes and are never confused with each other:

- **No documented endpoint** — the provider has no known usage-window endpoint at all (no entry in `scripts/providers/quota-endpoints.json`), so no network probe is even attempted: `not reported by provider`.
- **Probe failed** — an endpoint exists, but *this* probe timed out or errored (network, auth, or a bad response): `probe failed: <reason>`.

## Account-level block

A fully-suspended account additionally renders a distinct `ACCOUNT BLOCKED` statement, in addition to (never instead of) any windows it still carries from before the suspension.

## De-duplication

`deepseek`, `kimi-deepseek`, and `pi-deepseek` authenticate as the same real account via the same real API key (see §4.2) — `quota` probes that one account once and shows it once, listing every alias name that maps to it, rather than printing three separate rows with three separate (but identical) numbers.

## Example: a funded provider

The exact set of providers with a documented endpoint depends on `scripts/providers/quota-endpoints.json`; the aliases below illustrate the output format, not a live snapshot of any one host's configuration.

```
openrouter  (alias: openrouter)
  subscription  24.00 used / 76.00 left of 100.00 credits
(76.00% left)   [GREEN]

zai-coding-plan  (alias: zai-coding-plan)
  daily         850 used / 150 left of 1000 requests   (15.00%
left)   [YELLOW]

helixagent  (alias: helixagent)
  session       216204 used / 13172 left of 229376 tokens   (5.74%
left)   [RED]
```

### Example: honest absence — no documented endpoint, and a native account

```
groq  (alias: groq)
  –   not reported by provider

claude1  (native, plan tier: default_claude_max_20x)
  –   not reported by provider
```

Native accounts (`claudeN`, `kimiN`) always report this way today — no genuine live usage API exists yet for native OAuth accounts (per this feature's own `research.md` §7) — optionally annotated with the account's own cached plan-tier name when one is known, exactly as shown above for `claude1`.

### Performance

Probing is bounded-concurrency, not linear: total wall-clock time scales with `ceil(N / CMA_QUOTA_MAX_PARALLEL_PROBES)` batches of the per-provider timeout (`CMA_QUOTA_MAX_PARALLEL_PROBES` defaults to `8`), never with the raw account count. A single slow or unreachable provider degrades only its own row to `probe failed` and never delays any other row's result or the command's own exit. Cached results disclose their own age in seconds alongside the data.

This section, together with the feature's own `specs/002-provider-usage-command/quickstart.md`, is the complete documentation and validation surface for this command — there is no separate `quickstart/`-style tutorial directory for it, and none is needed.

---

## 15. Decision endpoints (Jev) — non-launchable records

`claude-providers list` has a second section below the agent rows:

```
## Decision endpoints (Jev) — not launchable; decision layer
```

These rows are **decision models** (Jev System-One endpoints), not chat models. They answer typed questions (`choice` / `noul` / `score`) via `POST <base>/v1/systemone` and have no chat-completions API — so they are registered with `classification: "decision"` and `"alias": null`, and the emission paths that would turn a record into `<id>` / `kimi-<id>` / `pi-<id>` aliases skip them by construction. **There is no `jev-local` alias to launch, and there never will be**; the `jev-*` namespace is reserved for the decision layer and rejected for account/provider alias names.

What decision records DO get:

- a health row in `status.json` (same `status`/`reason` shape as providers, plus `"classification": "decision"`), so endpoint health is visible in `claude-providers status`/`list`;
- a ranked entry in the endpoint cache `~/.local/share/claude-multi-account/providers/jev-endpoints.json` (written by `claude-providers sync` / `sync-jev`), which the launch wrappers read to export the `JEV_*`/`TYPESAFE_*` environment and print the `Jev decision layer: ACTIVE (…)` MOTD on every alias launch.

Sources and ranking: local llmctl decision gateway at `127.0.0.1:8095` (10) > LAN pins in `scripts/providers/jev.json` (20; LAN or remote cloud hosts you declare) > OpenRouter (30) > TypeSafe hosted (40); override with `CMA_JEV_PREFER`. Hosted sources are detected by key-var NAME presence in the keys file — no network call; local/LAN sources require a live `/healthz` + `/v1/models` probe.

The decision layer built on these records (permission gate, MCP tools, installer, stats) is documented in docs/jev/Jev_User_Guide.md; the rationale for non-launchability and the shadow-first calibration policy is in docs/jev/Jev_FAQ.md.

---

# Governance context: hub engine, validation gate, decision models

- **Hub engine.** claude_toolkit is consumed by reference at `constitution/submodules/claude_toolkit` in a project that vendors the Helix constitution (depth 1, never initialised recursively). Its own-org dependencies are resolved flat by `scripts/resolve_deps.sh` and declared in `helix-deps.yaml` — never nested (Constitution §11.4.28(C), §11.4.31). Placement diagram.
- **Local models (`llmctl-<profile>` aliases).** Their known-answer validation is llmctl's gate, Constitution §11.4.277 (`LLMCTL_VALIDATE_STRICT=1`, `llmctl require-validated <profile>`); Vision QA (§11.4.278) relies on it. See llmctl user guide §5a.
- **Decision models (Jev) are not aliases.** Constitution §11.4.279 treats a decision model as a typed-answer layer (choice / noul / score), separate from chat. Here they carry `classification: decision`, `alias: null`, and never produce a launchable alias. The llmctl decide gateway is HTTPS with a mandatory bearer key, which the detector, gate and MCP server speak (v1.32.0). See the Jev guide and the decide-gateway flow.
- **Alias, provider and model stay consistent.** An alias is only valid for the provider and model it was verified against. Re-run `claude-providers sync` after any provider or llmctl change.